WHEN DATASETS CHANGE SILENTLY: PITFALLS OF UNDOCUMENTED DATASET EVOLUTION IN ML REPOSITORIES

Anonymous authors
Paper under double-blind review

ABSTRACT

Machine learning datasets frequently undergo updates, corrections, and deprecations throughout their lifecycle, yet current repository practices lack systematic mechanisms to document and communicate these changes to downstream users. This creates reproducibility challenges—researchers may unknowingly use different dataset versions—and prevents informed decision-making about model retraining or result interpretation. We implement and evaluate a lightweight framework for automated dataset change detection and impact analysis, using hash-based row-level comparison to identify additions, deletions, modifications, and schema changes between dataset versions. Through controlled experiments on synthetic datasets with ground-truth changes, we reveal a critical pitfall: the detectability of modifications depends heavily on how many columns are examined per row, with sparse column-sampling strategies (25% of columns) achieving only 0.807 modifications F1 compared to 1.0 for full-coverage strategies. We further find that temporal drift induces high false-positive rates ($\approx$0.95) for most drift types, meaning automated change detectors may incorrectly flag benign distribution shifts as data errors. Our results highlight that building reliable change documentation infrastructure is significantly harder in practice than it appears, and we discuss the concrete implications for dataset lifecycle management in ML repositories.

1 INTRODUCTION

The reproducibility crisis in machine learning is well-documented (**??**), yet one underappreciated contributor is the silent evolution of datasets. When a dataset on a platform like HuggingFace (**?**) is updated—rows corrected, labels revised, columns added—downstream users may be unaware. Models trained on different versions of the "same" dataset produce incomparable results, and benchmarks shift without explanation. Yet empirical studies show that most ML repositories lack proper dataset versioning infrastructure (**?**), and existing documentation standards such as Datasheets for Datasets (**?**) and Model Cards (**?**) address initial creation rather than ongoing evolution.

We set out to build and evaluate an automated change documentation system to fill this gap. The core idea is straightforward: compare two dataset versions, detect what changed (rows added, deleted, modified; schema altered), generate a structured change log, and flag statistically significant shifts. In theory, this is a solved problem—hash-based comparison is well-understood (**?**). In practice, we find that the problem is considerably harder than expected, with multiple non-obvious failure modes. Our contributions are: (1) a controlled evaluation of hash-based change detection under varying column-sampling strategies, revealing that sparse column coverage causes systematic false negatives; (2) an analysis of temporal drift robustness, showing that most drift types induce near-total false-positive rates ($\approx$0.95); (3) a scalability analysis confirming near-linear computational complexity ($O(n^{0.91})$) with stable accuracy; and (4) a multi-version transitive detection analysis showing modest but increasing degradation across version chains.

2 Related Work

Prior work on dataset documentation has focused primarily on static artifacts created at dataset release time. Datasheets for Datasets (**?**) and Model Cards (**?**) provide frameworks for communicating dataset and model properties at creation, but offer no mechanism for documenting subsequent changes. Large-scale analyses of dataset cards on HuggingFace (**?**) reveal substantial heterogeneity in documentation completeness, while automated approaches to generating initial documentation (**??**) address the creation problem but not longitudinal updates. Technical provenance tracking (**??**) captures computational lineage in ML pipelines but does not produce human-readable differential documentation of what changed between versions and why. Concept drift detection (**??**) monitors model performance degradation due to changing data distributions, but operates at the model level rather than documenting specific dataset-level changes. Distribution shift quantification (**?**) provides statistical tools for measuring data similarity but does not address the documentation and communication problem. Dataset versioning tools (**?**) provide version control infrastructure but lack automated impact analysis, and the relationship between dataset properties and downstream model performance (**?**) motivates the need for systematic change documentation. Our work uniquely addresses the gap between these areas: automated detection and documentation of *what* changed between dataset versions, with analysis of the conditions under which such detection succeeds or fails.

3 Method

Our framework compares two versions of a tabular dataset. Given datasets D_1 and D_2 sharing a primary key column, we detect four change types: (1) **additions**—rows in D_2 absent from D_1; (2) **deletions**—rows in D_1 absent from D_2; (3) **modifications**—rows present in both versions but with differing values; (4) **schema changes**—columns added or removed. Modification detection uses row-level MD5 hashing: for each common row identifier, we concatenate string representations of all common-column values and compare hashes. This approach is exact (zero false positives by construction) but its recall depends critically on which columns are included in the hash. If only a subset of columns is compared, modifications confined to excluded columns will be missed—a fundamental limitation we investigate systematically. The framework generates structured change logs enumerating detected changes by type, along with summary statistics. We evaluate detection quality against ground-truth changes introduced synthetically, measuring precision, recall, and F1 separately for each change type.

4 Experiments

We generate synthetic base datasets of 1,000 rows with 6 feature columns (mix of numeric and categorical) plus an ID and target column. We introduce ground-truth changes under controlled configurations and evaluate detection accuracy against known ground truth, varying: the column-selection strategy; dataset size (100–10,000 rows); modification depth (columns changed per modified row); temporal drift type and magnitude; detection algorithm; and temporal latency between versions.

Column Coverage is Critical for Modification Detection. Figure 1 shows that the choice of which columns to examine when hashing rows has a dramatic effect on modification recall. Six of eight strategies achieve perfect accuracy (1.0) and modifications F1 (1.0): `fixed_two`, `all_columns`, `random_subset_50`, `numeric_only`, `categorical_only`, and `weighted_numeric`. However, `random_subset_25` achieves only 0.891 overall accuracy and 0.807 modifications F1, while `single_random` achieves 0.921 accuracy and 0.867 modifications F1. Critically, precision remains 1.0 for all strategies—all errors are false negatives (missed modifications), never false positives. This reveals a fundamental practical pitfall: if a change detection system examines only a sparse subset of columns for efficiency, it will systematically miss modifications to excluded columns. The 50% random subset closes the gap entirely, but the 25% subset does not, with direct implications for scalable change detection on wide datasets.

Modification Depth and Algorithm Comparison. Figure 2 reveals two complementary failure modes. Single-column modifications (depth=1) are the hardest to detect, yielding only 0.805 mean accuracy and 0.64 recall—precisely the scenario most common in real-world dataset corrections,

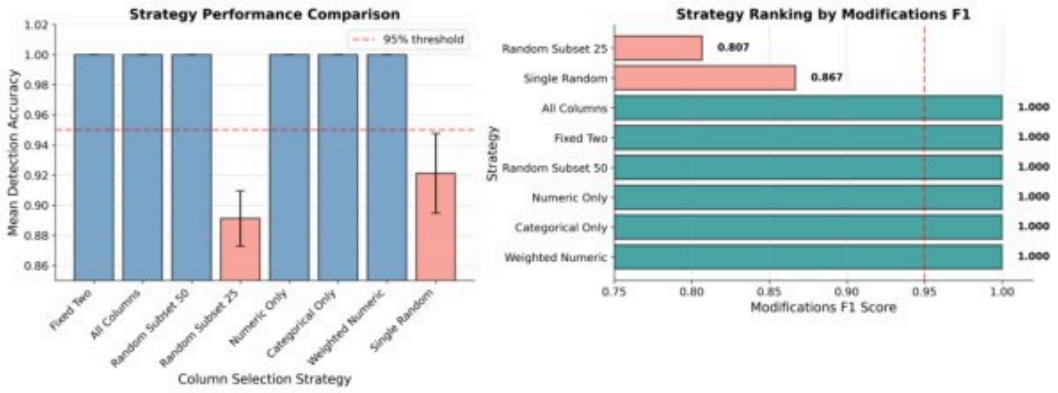
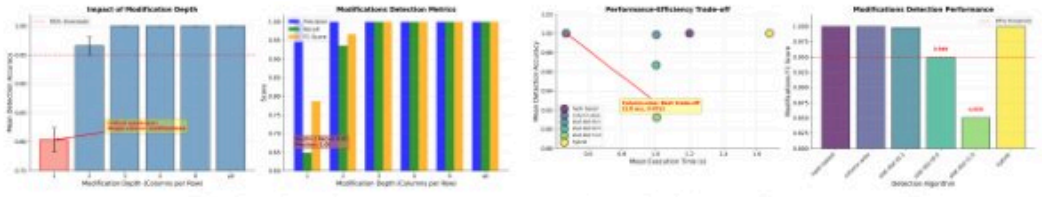


Figure 1: Column selection strategy performance. Left: mean detection accuracy—six strategies achieve 1.0, while `random_subset_25` (0.891) and `single_random` (0.921) fall below the 0.95 threshold. Right: modifications F1 ranking—sparse column sampling causes systematic false negatives, with `random_subset_25` reaching only 0.807 and `single_random` reaching 0.867. Precision is 1.0 for all strategies; all errors are false negatives.



(a) Modification depth impact (b) Algorithm performance vs. efficiency

Figure 2: (a) Single-column modifications (depth=1) are a critical weakness—accuracy drops to ≈0.80 with recall ≈0.64, the most common real-world correction scenario. Depth $\geq$ 2 restores near-perfect detection. (b) Hash-based and column-wise algorithms achieve 1.0 accuracy at lowest execution time (≈0.5s); statistical distance degrades with higher thresholds (`stat_dist_tl.0`: 0.913 accuracy, 0.850 F1); hybrid is accurate but slow (≈1.7s).

where a single erroneous field is fixed. At depth $\geq$ 2, accuracy recovers to near 1.0. Comparing six detection algorithms, hash-based and column-wise methods achieve 1.0 accuracy at the lowest execution times (≈0.5s), while statistical distance methods degrade with increasing threshold: `stat_dist_tl.0` drops to 0.913 accuracy and 0.850 modifications F1. The hybrid method achieves 1.0 accuracy but at the highest cost (≈1.7s). The column-wise approach offers the best trade-off: perfect accuracy at minimal computational cost.

Temporal Drift Induces Catastrophic False Positives. A key failure mode for deployed change detection systems is temporal drift: gradual, benign distributional changes that should *not* be flagged as data errors. Figure 3 shows that for five of six drift types (distribution shift, variance expansion, range expansion, gradual noise injection, mixed drift), the false positive rate under drift is ≈0.95 across all magnitudes—the detector incorrectly flags nearly every drifted row as a modification. Only category frequency shift achieves meaningfully lower false positives (≈0.68–0.71). The drift robustness score is correspondingly ≈0.04 for most drift types and ≈0.31 for category frequency shift. This stark finding underscores a serious practical limitation: a change detection system deployed on a live dataset will generate overwhelming false alarms whenever the underlying data distribution shifts gradually, making it nearly impossible to distinguish genuine data errors from natural evolution without additional statistical modeling.

Scalability and Multi-Version Analysis. Figure 4 confirms that detection accuracy remains stable at 1.0 across dataset sizes from 100 to 10,000 rows, with computational complexity scaling as $O(n^{0.91})$—near-linear, suitable for practical deployment. The multi-version analysis reveals that comparing $v_1 \rightarrow v_3$ directly (rather than maintaining pairwise $v_1 \rightarrow v_2$ and $v_2 \rightarrow v_3$ comparisons)

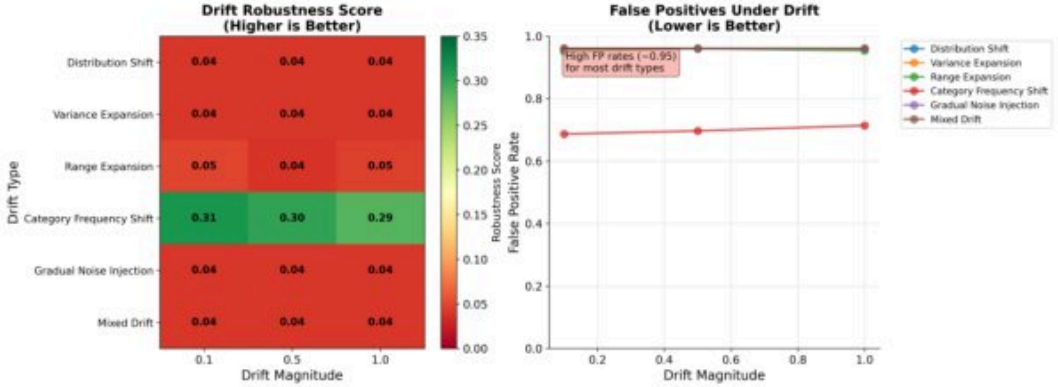
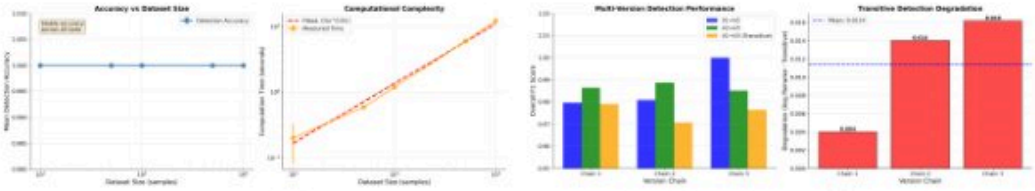


Figure 3: Temporal drift robustness. Heatmap (left): robustness score is near zero (≈0.04) for five of six drift types; category frequency shift reaches ≈0.31. Line plot (right): false positive rates under drift are ≈0.95 for most types across all magnitudes, with category frequency shift lower (≈0.68–0.71). The detector cannot reliably distinguish genuine data errors from benign distributional changes.



(a) Scalability (b) Multi-version transitive detection

Figure 4: (a) Detection accuracy is stable at 1.0 across all dataset sizes (100–10,000 rows); computation scales as $O(n^{0.91})$. (b) Transitive detection (comparing $v_1 \rightarrow v_3$ directly) shows monotonically increasing degradation (0.004, 0.014, 0.016 across three version chains; mean 0.011), recommending pairwise comparisons over transitive inference.

introduces small but systematically increasing degradation: 0.004, 0.014, and 0.016 across three version chains (mean 0.011). While modest in absolute terms, the monotonically increasing trend suggests that transitive inference compounds errors across longer version chains, recommending pairwise comparison as the preferred strategy.

5 Conclusion

We evaluated a hash-based automated change detection framework for ML dataset repositories, uncovering several non-obvious pitfalls. The system achieves near-perfect detection under ideal conditions (full column coverage, immediate comparison, no drift), but fails in practically important scenarios: sparse column sampling causes systematic false negatives for single-field modifications; temporal drift induces near-total false-positive rates; and transitive detection across long version chains degrades monotonically. These findings suggest that building reliable dataset change documentation is harder than it appears, and naive deployment of even technically sound detection methods can mislead practitioners. Future work should develop statistical models of expected data evolution to distinguish genuine errors from benign drift, smarter column prioritization strategies for wide datasets, and standardized evaluation benchmarks on real (not synthetic) dataset version histories.

REFERENCES

SUPPLEMENTARY MATERIAL

A CHANGE MAGNITUDE INTENSITY

Figure 5 shows that detection accuracy and modifications metrics are uniformly 1.0 across subtle, moderate, and drastic change magnitudes when column coverage is complete. This confirms that the magnitude of value changes does not affect detectability for hash-based methods—any change to a hashed value, however small, produces a different hash. The practical challenge lies in column coverage (Section 4), not change magnitude per se.

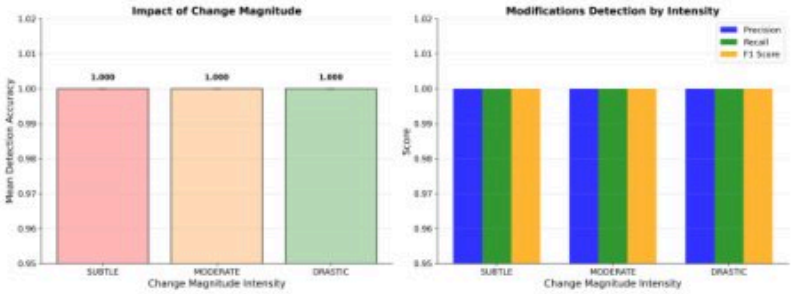


Figure 5: Change magnitude intensity has no effect on detection accuracy or modifications metrics under full column coverage. All metrics remain at 1.0 across subtle, moderate, and drastic change magnitudes, confirming that any value change, however small, produces a different hash.

B ALGORITHM METRICS HEATMAP

Figure 6 provides per-configuration detail for all six detection algorithms across four metrics: overall accuracy, precision, recall, and F1 score. Precision is uniformly 1.0 for all algorithms and configurations. Recall degrades for statistical distance methods with higher thresholds—`statistical_dist_t1.0` achieves recall 0.925–0.947 and `statistical_dist_t0.5` achieves recall 0.969–0.983—which propagates to reduced F1 (0.956–0.970 and 0.983–0.991, respectively). Overall accuracy for `statistical_dist_t1.0` ranges from 0.900–0.926. Hash-based, column-wise, and hybrid methods achieve perfect scores across all configurations and metrics.

C TEMPORAL DRIFT DETAILED COMPARISON

Figure 7 provides per-drift-type breakdown of robustness scores and modifications F1 as a function of drift magnitude, using a 2×3 subplot grid. For five drift types (distribution shift, variance expansion, range expansion, gradual noise injection, mixed drift), both drift robustness (blue lines) and modifications F1 (red lines) are essentially flat at $\approx$0.04–0.05 and $\approx$0.06–0.07 respectively, showing no meaningful variation across magnitudes. Category frequency shift is the sole exception, with robustness $\approx$0.28–0.30 with a slight downward trend as drift magnitude increases. Notably, robustness scores do not improve with smaller drift magnitudes for continuous drift types, confirming that even subtle distributional changes are sufficient to trigger near-maximal false positive rates.

D COLUMN STRATEGY STABILITY ACROSS CONFIGURATIONS

Figure 8 shows column selection strategy detection accuracy across three experiment configurations. Six strategies (Fixed Two, All Columns, Random Subset 50, Numeric Only, Categorical Only, Weighted Numeric) remain flat at 1.0 across all configurations. Random Subset 25 increases monotonically from 0.867 to 0.899 to 0.912, while Single Random shows a V-shaped pattern (0.917

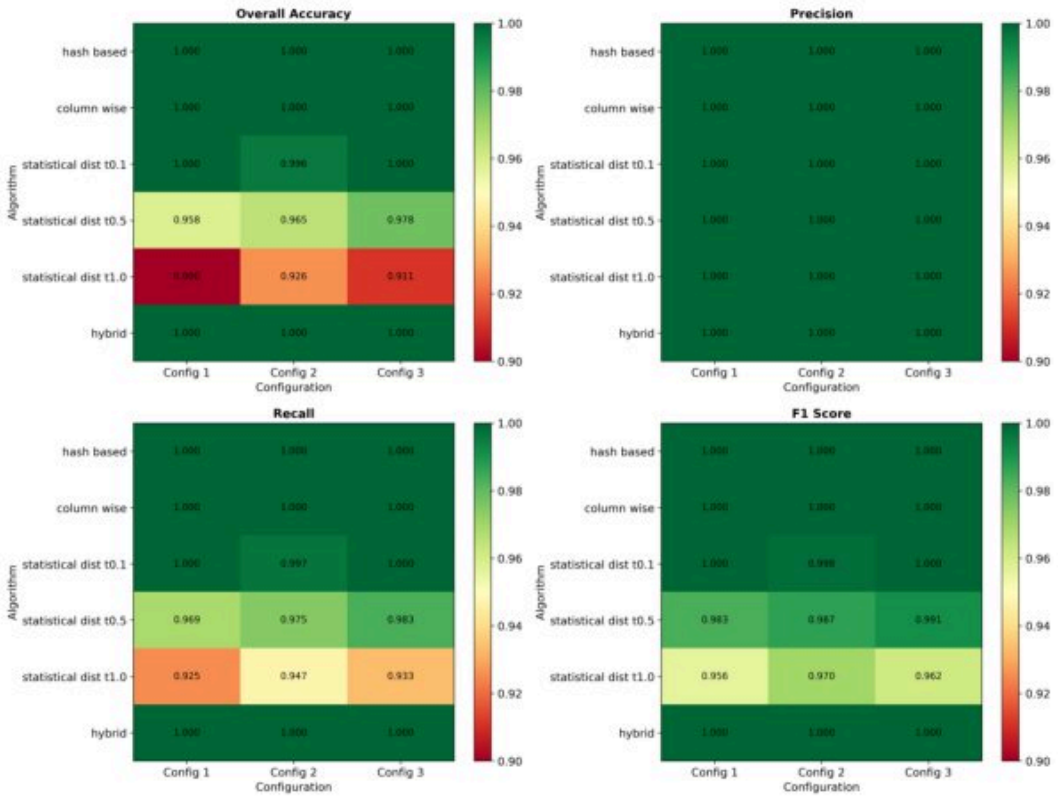


Figure 6: Per-configuration algorithm performance heatmaps for all four metrics (overall accuracy, precision, recall, F1). Precision is uniformly 1.0 across all algorithms; recall and F1 degrade for statistical distance algorithms with higher thresholds (`stat_dist_t1.0`: recall 0.925–0.947, F1 0.956–0.970). Hash-based, column-wise, and hybrid methods achieve perfect scores.

$\rightarrow$ 0.891 $\rightarrow$ 0.955), reflecting instability from non-deterministic column selection—a risk for production systems where reproducible detection is essential for auditing.

E Temporal Latency Analysis

Figure 9 shows detection performance as a function of temporal latency across three views. The left panel (accuracy vs. change rounds) shows a non-monotonic decline from 0.984 (immediate, 1 round) to 0.952 (extended term, 10 rounds), with a dip at long term ($\approx$0.957) followed by partial recovery at medium term ($\approx$0.975). The middle panel shows modifications F1 by latency: high for immediate (0.985), short term (0.983), and medium term (0.985), then dropping at long term (0.960) and extended term (0.963). The right panel shows transitive detection degradation by version chain (0.004, 0.014, 0.016; mean 0.011), consistent with the main text analysis. Together, these results suggest that change detection systems should be run frequently and maintain pairwise version comparisons rather than relying on transitive inference.

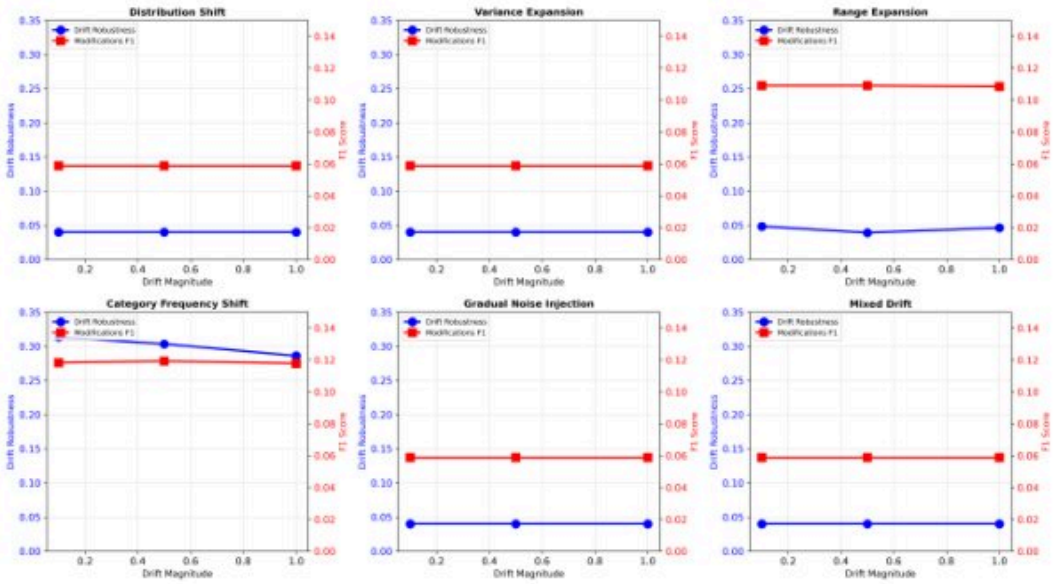


Figure 7: Per-drift-type robustness scores (blue) and modifications F1 (red) across drift magnitudes (2×3 subplots). Five of six drift types show flat, near-zero robustness (≈0.04–0.05) across all magnitudes. Category frequency shift reaches ≈0.28–0.30 but degrades slightly with increasing magnitude. No drift type achieves robustness above 0.35.

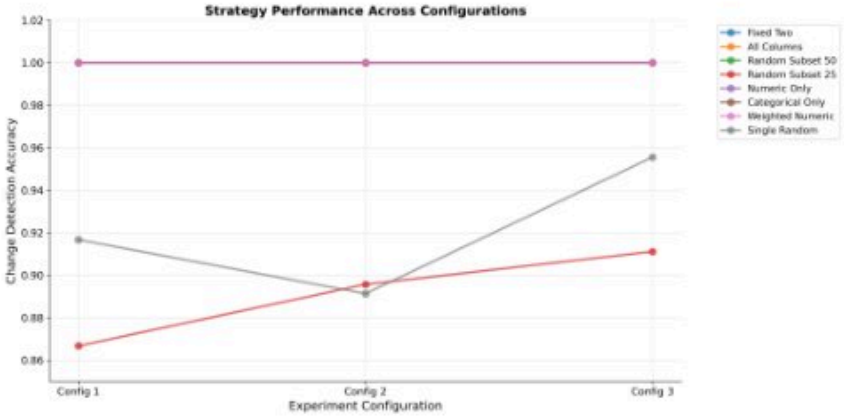


Figure 8: Column selection strategy detection accuracy across three experiment configurations. Six strategies remain flat at 1.0. Random Subset 25 improves monotonically (0.867 → 0.899 → 0.912) while Single Random is unstable (0.917 → 0.891 → 0.955), illustrating the risk of non-deterministic column sampling.

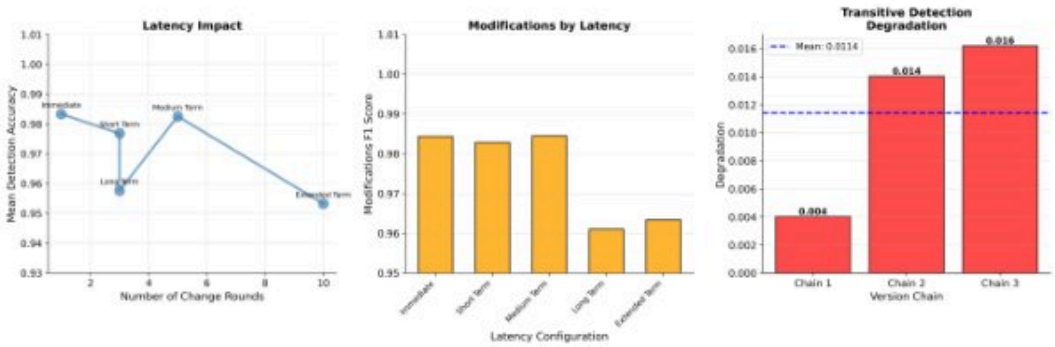


Figure 9: Latency and multi-version analysis. Left: mean detection accuracy vs. change rounds—non-monotonic decline from 0.984 (immediate) to 0.952 (extended term). Middle: modifications F1 by latency—drops from ≈0.985 at short latency to ≈0.960–0.963 at long/extended latency. Right: transitive detection degradation by version chain (0.004, 0.014, 0.016; mean 0.011).